

VPC – Transit Gateway

Class No:17

AWS Transit Gateway

An AWS Transit Gateway acts as a central hub (a "cloud router") that connects your Amazon Virtual Private Clouds (VPCs) and on-premises networks. As your cloud footprint grows, managing individual connections between every VPC becomes nearly impossible; the Transit Gateway simplifies this by providing a single point of management.

Primary Purpose

The main goal of a Transit Gateway is to eliminate complex peering topologies.

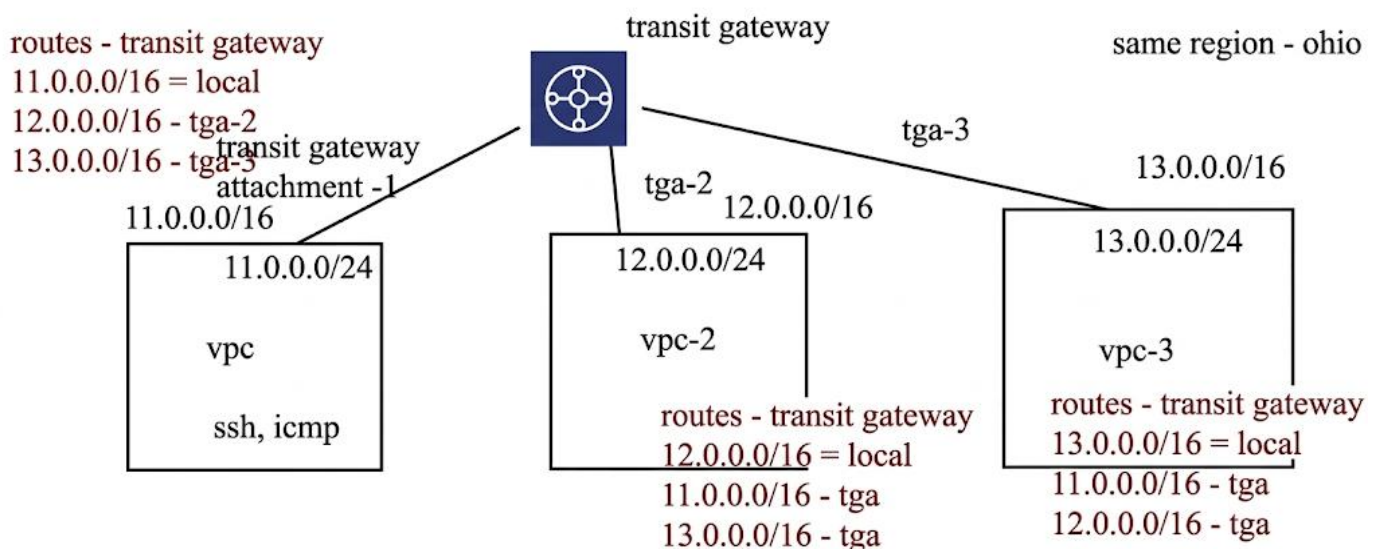
- **Centralized Connectivity:** Instead of creating a "mesh" of VPC Peering connections, you connect each VPC or VPN to the Transit Gateway.
- **Hybrid Cloud Hub:** It bridges the gap between your AWS environment and your on-premises data centers via Direct Connect or Site-to-Site VPNs.
- **Simplified Routing:** It manages how traffic flows between all connected entities using route tables, much like a traditional physical router.
- **Cross account support:** Using **AWS Resource Access Manager (RAM)**, you can share a single Transit Gateway across multiple AWS accounts within your organization.

Transit Gateway vs. VPC Peering

To understand the advantage, consider the math of a full mesh network:

- With Peering: For 10 VPCs, you need 45 separate peering connections.
- With Transit Gateway: For 10 VPCs, you need only 10 attachments.

Architecture of Transit gateway:



Hands on: Creating transit gateway between three VPCs

Step no:1 Create three VPCs for different zones and create EC instances in that respective VPC respectively.

I am creating in Osaka region, 3 VPCs& EC2 instances in the 3 different AZs such as northeast -3a, northeast -3b, northeast -3c.

Create connection between them like normally how we create a connection between our VPC and add those configurations in the EC2.

Ensure we are creating public EC2 and public route tables with internet gateway for internet connectivity.

VPCs | VPC encryption controls

Your VPCs (4) [Info](#)

☐	Name
☐	-
☐	Osaka-VPC-1
☐	Osaka-VPC-2
☐	Osaka-VPC-3

Subnets (6) [Info](#)

☐	Name
☐	Public-Subnet-1
☐	-
☐	Public-Subnet-3
☐	-
☐	Public-Subnet-2
☐	-

Route tables (7) [Info](#)

☐	Name
☐	-
☐	-
☐	-
☐	-
☐	-
☐	My-Rt1
☐	My-Rt2
☐	My-Rt3

Security Groups (7) [Info](#)

Security group name
default
My-sg1
My-sg3
default
default
My-sg2

Internet gateways (4) [Info](#)

☐	Name
☐	-
☐	My-Igw1
☐	My-Igw2
☐	My-Igw3

Instances (3) [Info](#)

Saved filter sets
Choose filter set ▼

☐	Name ✎	Instance ID	Instance state	Instance type
☐	My-instance1	i-0047a506b0a0acc65	✓ Running ⚙ ⌵	t3.micro
☐	My-instance3	i-0b1bf0e6b82d53fcb	⌚ Pending ⚙ ⌵	t3.micro
☐	My-instance2	i-0397fbe192f515ec4	✓ Running ⚙ ⌵	t3.micro

Step no:2 On the VPC page scroll down → choose Transit gateway → create gateway → Name: Osaka Tg1 → Keep the default configurations → create gateway.

(Only one transit gateway)

Create transit gateway [Info](#)

A transit gateway (TGW) is a network transit hub that interconnects attachments (VPNs and VPCs) within the same AWS account or across AWS accounts.

Details - optional

Name tag
Creates a tag with the key set to Name and the value set to the specified string.

Osaka-Tg1

Description [Info](#)
Set the description of your transit gateway to help you identify it in the future.

Osaka-Tg1

Configure the transit gateway

Amazon side Autonomous System Number (ASN) [Info](#)

ASN

☒ DNS support [Info](#)

☐ Security Group Referencing support [Info](#)

☒ VPN ECMP support [Info](#)

☒ Default route table association [Info](#)

☒ Default route table propagation [Info](#)

☐ Multicast support [Info](#)

Configure cross-account sharing options

☐ Auto accept shared attachments [Info](#)

Transit gateway CIDR blocks

CIDR - optional [Info](#)

Q 10.0.0.0/24

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional
Q Name X	Q Osaka-Tg1 X Remove

[Add new tag](#)

You can add up to 49 more tags.

[Cancel](#) [Create transit gateway](#)

Transit gateways (4) [Info](#)

Q Find transit gateway by attribute or tag

☐	Name ✎	Transit gateway ID
☐		tgw-04e96c721de00d5ab
☐	Osaka-Tg1	tgw-08e34ace1b43ae9de

Step no:3 Now scroll down → Choose transit gateway attachment → Create 3 attachment for 3 respective VPCs.

VPC1:

Name: **Osaka-TGattachment-1**

Transit gateway ID is same for all

Attachment Type: **VPC**

VPC Id: **ID of Osaka VPC1**

Respective Subnet Region

Create Transit Gateway

Create transit gateway attachment [Info](#)

A transit gateway (TGW) is a network transit hub that interconnects attachments (VPCs and VPNs) within the same AWS account or across AWS accounts.

Details

Name tag - optional

Creates a tag with the key set to Name and the value set to the specified string.

Osaka-TGattachment1

Transit gateway ID

[Info](#)

tgw-08e34ace1b43ae9de

Attachment type

[Info](#)

VPC

VPC attachment

Select and configure your VPC attachment.

☒ DNS support [Info](#)

☒ Security Group Referencing support [Info](#)

☐ IPv6 support [Info](#)

☐ Appliance Mode support [Info](#)

VPC ID

Select the VPC to attach to the transit gateway.

vpc-0ddca4ad66201f12b

Subnet IDs

[Info](#)

Select the subnets in which to create the transit gateway VPC attachment.

☒ ap-northeast-3a

subnet-0ec5ad70290d3b5e4

☐ ap-northeast-3b

No subnet available

☐ ap-northeast-3c

No subnet available

subnet-0ec5ad70290d3b5e4 X

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name X

Value - optional

Q Osaka-TGattachment1 X

Remove

Add new tag

You can add up to 49 more tags.

Cancel

Create transit gateway attachment

VPC2:

Name: **Osaka-TGattachment-2**

Transit gateway ID is same for all

Attachment Type: **VPC**

VPC Id: **ID of Osaka VPC2**

Respective Subnet Region

Create Transit Gateway

Create transit gateway attachment [Info](#)

A transit gateway (TGW) is a network transit hub that interconnects attachments (VPCs and VPNs) within the same AWS account or across AWS accounts.

Details

Name tag - optional

Creates a tag with the key set to Name and the value set to the specified string.

Osaka-TGAttachment2

Transit gateway ID

[Info](#)

tgw-08e34ace1b43ae9de

Attachment type

[Info](#)

VPC

VPC attachment

Select and configure your VPC attachment.

☒ DNS support [Info](#)

☒ Security Group Referencing support [Info](#)

☐ IPv6 support [Info](#)

☐ Appliance Mode support [Info](#)

VPC ID

Select the VPC to attach to the transit gateway.

vpc-0bec5c8bdaf715e9a

Subnet IDs

[Info](#)

Select the subnets in which to create the transit gateway VPC attachment.

☐ ap-northeast-3a

No subnet available

☒ ap-northeast-3b

subnet-0171f1506eb217389

☐ ap-northeast-3c

No subnet available

subnet-0171f1506eb217389 X

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name X

Value - optional

Q Osaka-TGAttachment2 X

Remove

Add new tag

You can add up to 49 more tags.

Cancel

Create transit gateway attachment

VPC3:

Name: **Osaka-TGattachment-3**

Transit gateway ID is same for all

Attachment Type: **VPC**

VPC Id: **ID of Osaka VPC3**

Respective Subnet Region

Create Transit Gateway

Create transit gateway attachment [Info](#)

A transit gateway (TGW) is a network transit hub that interconnects attachments (VPCs and VPNs) within the same AWS account or across AWS accounts.

Details

Name tag - optional

Creates a tag with the key set to Name and the value set to the specified string.

Osaka-TGAttachment3

Transit gateway ID

tgw-08e34ace1b43ae9de

Attachment type

VPC

VPC attachment

Select and configure your VPC attachment.

☒ DNS support [Info](#)

☒ Security Group Referencing support [Info](#)

☐ IPv6 support [Info](#)

☐ Appliance Mode support [Info](#)

VPC ID

Select the VPC to attach to the transit gateway.

vpc-060f091e071a2448a

Subnet IDs

Select the subnets in which to create the transit gateway VPC attachment.

☐ ap-northeast-3a No subnet available

☐ ap-northeast-3b No subnet available

☒ ap-northeast-3c subnet-0a827fde68969b5b

subnet-0a827fde68969b5b

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q Osaka-TGAttachment3

Remove

Add new tag

You can add up to 49 more tags.

Cancel

Create transit gateway attachment

Step no:4 Now go to the Route Table → choose My-RT1 of the VPC1 → Select → Routes → add the CIDR of other two VPCs as,

12.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC1:Osaka-TGAttachment1

13.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC12:Osaka-TGAttachment1

Route tables > rtb-04b9b8550c613f738 > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
11.0.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Internet Gateway	Active	No	CreateRoute
Q 12.0.0.0/16	Transit Gateway	-	No	CreateRoute
Q 13.0.0.0/16	Transit Gateway	-	No	CreateRoute

Cancel

Preview

Save changes

Similarly, go to the Route Table → choose My-RT2 of the VPC2 → Select → Routes → add the CIDR of other two VPCs as,

11.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC1:Osaka-TGAttachment2

13.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC3:Osaka-TGAttachment2

Edit routes

Destination	Target	Status	Propagated	Route Origin	
12.0.0.0/16	local	Active	No	CreateRouteTable	
0.0.0.0/0	Internet Gateway	Active	No	CreateRoute	Remove
13.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove
11.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove

Use: "tgw-"
tgw-08e34ace1b43ae9de (Osaka-TGAttachment2)

Cancel Preview Save changes

Similarly, go to the Route Table → choose My-RT3 of the VPC3 → Select → Routes → add the CIDR of other two VPCs as,

11.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC1:Osaka-TGAttachment3

12.0.0.0/16 → Transit gateway → choose the Transit gateway attachment created for the VPC2:Osaka-TGAttachment3

Edit routes

Destination	Target	Status	Propagated	Route Origin	
13.0.0.0/16	local	Active	No	CreateRouteTable	
0.0.0.0/0	Internet Gateway	Active	No	CreateRoute	Remove
11.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove
12.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove

Use: "tgw-08e34ace1b43ae9de"
tgw-08e34ace1b43ae9de (Osaka-TGAttachment3)

Cancel Preview Save changes

Step no:5 now Check the route tables, whether they have all the CIDR range of the local VPC and other two VPCs.

rtb-04b9b8550c613f738 / My-Rt1

You can now check network connectivity with Reachability Analyzer

Details

Route table ID: rtb-04b9b8550c613f738

VPC: vpc-08dca4a66201f12b | Osaka-VPC-1

Owner ID: 314383175192

Routes (4)

Destination	Target
0.0.0.0/0	tgw-08f9d36c9a05c005d
11.0.0.0/16	local
12.0.0.0/16	tgw-08e34ace1b43ae9de
13.0.0.0/16	tgw-08e34ace1b43ae9de

rtb-0f9e8f092c721e90e / My-Rt2

You can now check network connectivity with Reachability Analyzer

Details

Route table ID: rtb-0f9e8f092c721e90e

VPC: vpc-08ecdc68daf715e9a | Osaka-VPC-2

Owner ID: 314383175192

Routes (4)

Destination	Target
0.0.0.0/0	tgw-099335b0519c16335
11.0.0.0/16	tgw-08e34ace1b43ae9de
12.0.0.0/16	local
13.0.0.0/16	tgw-08e34ace1b43ae9de

rtb-0debfa51752941b53 / My-Rt3

You can now check network connectivity with Reachability Analyzer

Details

Route table ID: rtb-0debfa51752941b53

VPC: vpc-06f091e071a248a | Osaka-VPC-3

Owner ID: 314383175192

Routes (4)

Destination	Target
0.0.0.0/0	tgw-0e1f2027a41ac5e1
11.0.0.0/16	tgw-08e34ace1b43ae9de
12.0.0.0/16	tgw-08e34ace1b43ae9de
13.0.0.0/16	local

Step no:6 Now go our EC2 instances and connect all the tree (duplicate tabs to see all the three connect pages).

Instances (3) Info

Saved filter sets
Choose filter set ▼

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	My-instance1	i-0047a506b0a0acc65	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-3a
☐	My-instance3	i-0b1bf0e6b82d53fcb	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-3c
☐	My-instance2	i-0397fbe192f515ec4	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-3b

Open the Connect of instance one → ping the private Ip of the other two instances. → It will transmit packets.

Command : **ping private_ip_address_of_other_EC2**

Here the private ip of instance-2 : **12.0.0.190** and private Ip of instance-3:**13.0.0.17**

```
[ec2-user@ip-11-0-0-57 ~]$ ping 12.0.0.190
PING 12.0.0.190 (12.0.0.190) 56(84) bytes of data.
64 bytes from 12.0.0.190: icmp_seq=1 ttl=126 time=1.30 ms
64 bytes from 12.0.0.190: icmp_seq=2 ttl=126 time=0.930 ms
64 bytes from 12.0.0.190: icmp_seq=3 ttl=126 time=0.938 ms
64 bytes from 12.0.0.190: icmp_seq=4 ttl=126 time=0.945 ms
^C
--- 12.0.0.190 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.930/1.029/1.304/0.158 ms
[ec2-user@ip-11-0-0-57 ~]$ ping 13.0.0.17
PING 13.0.0.17 (13.0.0.17) 56(84) bytes of data.
64 bytes from 13.0.0.17: icmp_seq=1 ttl=126 time=1.36 ms
64 bytes from 13.0.0.17: icmp_seq=2 ttl=126 time=0.935 ms
64 bytes from 13.0.0.17: icmp_seq=3 ttl=126 time=0.909 ms
64 bytes from 13.0.0.17: icmp_seq=4 ttl=126 time=0.935 ms
^C
--- 13.0.0.17 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.909/1.035/1.364/0.189 ms
[ec2-user@ip-11-0-0-57 ~]$
```

i-0047a506b0a0acc65 (My-instance1)

PublicIPs: 15.168.164.154 PrivateIPs: 11.0.0.57

Similarly,

Open the Connect of instance 2 → ping the private Ip of the other two instances. → It will transmit packets.

Command : **ping private_ip_address_of_other_EC2**

Here the private ip of instance-1 : **11.0.0.57** and private Ip of instance-3:**13.0.0.17**


```

_/m/'
Last login: Thu Apr  2 15:40:09 2026 from 15.168.105.160
[ec2-user@ip-13-0-0-17 ~]$ ping 11.0.0.57
PING 11.0.0.57 (11.0.0.57) 56(84) bytes of data.
64 bytes from 11.0.0.57: icmp_seq=1 ttl=126 time=1.34 ms
64 bytes from 11.0.0.57: icmp_seq=2 ttl=126 time=0.906 ms
64 bytes from 11.0.0.57: icmp_seq=3 ttl=126 time=0.980 ms
64 bytes from 11.0.0.57: icmp_seq=4 ttl=126 time=0.916 ms
64 bytes from 11.0.0.57: icmp_seq=5 ttl=126 time=0.948 ms
64 bytes from 11.0.0.57: icmp_seq=6 ttl=126 time=0.930 ms
64 bytes from 11.0.0.57: icmp_seq=7 ttl=126 time=0.915 ms
^C64 bytes from 11.0.0.57: icmp_seq=8 ttl=126 time=0.990 ms

64 bytes from 11.0.0.57: icmp_seq=9 ttl=126 time=0.908 ms
64 bytes from 11.0.0.57: icmp_seq=10 ttl=126 time=1.03 ms
^C
--- 11.0.0.57 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9227ms
rtt min/avg/max/mdev = 0.906/0.985/1.337/0.123 ms
[ec2-user@ip-13-0-0-17 ~]$ ping 11.0.0.57
PING 11.0.0.57 (11.0.0.57) 56(84) bytes of data.
64 bytes from 11.0.0.57: icmp_seq=1 ttl=126 time=1.28 ms
64 bytes from 11.0.0.57: icmp_seq=2 ttl=126 time=0.990 ms
64 bytes from 11.0.0.57: icmp_seq=3 ttl=126 time=1.02 ms
64 bytes from 11.0.0.57: icmp_seq=4 ttl=126 time=0.932 ms
64 bytes from 11.0.0.57: icmp_seq=5 ttl=126 time=0.946 ms
^C

```

Similarly,

Open the Connect of instance 2 → ping the private Ip of the other two instances. → It will transmit packets

Command: **ping private_ip_address_of_other_EC2**

Here the private ip of instance-1: **11.0.0.57** and private Ip of instance-3:**13.0.0.17**

```

_/m/'
Last login: Thu Apr  2 15:39:03 2026 from 15.168.105.160
[ec2-user@ip-12-0-0-190 ~]$ ping 11.0.0.57
PING 11.0.0.57 (11.0.0.57) 56(84) bytes of data.
64 bytes from 11.0.0.57: icmp_seq=1 ttl=126 time=0.949 ms
64 bytes from 11.0.0.57: icmp_seq=2 ttl=126 time=0.918 ms
64 bytes from 11.0.0.57: icmp_seq=3 ttl=126 time=0.938 ms
64 bytes from 11.0.0.57: icmp_seq=4 ttl=126 time=0.931 ms
64 bytes from 11.0.0.57: icmp_seq=5 ttl=126 time=0.952 ms
64 bytes from 11.0.0.57: icmp_seq=6 ttl=126 time=0.912 ms
64 bytes from 11.0.0.57: icmp_seq=7 ttl=126 time=0.937 ms
64 bytes from 11.0.0.57: icmp_seq=8 ttl=126 time=0.921 ms
^C
--- 11.0.0.57 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7070ms
rtt min/avg/max/mdev = 0.912/0.932/0.952/0.013 ms
[ec2-user@ip-12-0-0-190 ~]$ ping 13.0.0.17
PING 13.0.0.17 (13.0.0.17) 56(84) bytes of data.
64 bytes from 13.0.0.17: icmp_seq=1 ttl=126 time=1.02 ms
64 bytes from 13.0.0.17: icmp_seq=2 ttl=126 time=0.478 ms
64 bytes from 13.0.0.17: icmp_seq=3 ttl=126 time=0.497 ms
64 bytes from 13.0.0.17: icmp_seq=4 ttl=126 time=0.501 ms
64 bytes from 13.0.0.17: icmp_seq=5 ttl=126 time=0.481 ms
^C

```

As we ping the private ip of the instances interchangeably and it works, it shows the transit gateway communication is successful.

Transit Gateway Peering Attachment

To connect two AWS Transit Gateways—whether they are in the same region or different regions—you use a **Transit Gateway Peering Attachment**. This allows you to route traffic between VPCs and on-premises networks connected to different Transit Gateways.

Think of Transit Gateway (TGW) Peering as a "bridge" between two separate hubs. Once peered, the resources attached to TGW-A can communicate with resources attached to TGW-B, provided the route tables are configured correctly.

Key Advantages of TGW Peering

- **Inter-Region Connectivity:** The most common way to build a global AWS network. Traffic stays on the private AWS global backbone, avoiding the public internet.
- **Data Encryption:** All inter-region traffic is automatically encrypted at the physical layer.
- **Granular Control:** You can use separate TGW Route Tables to control exactly which VPCs in Region A can talk to which VPCs in Region B.

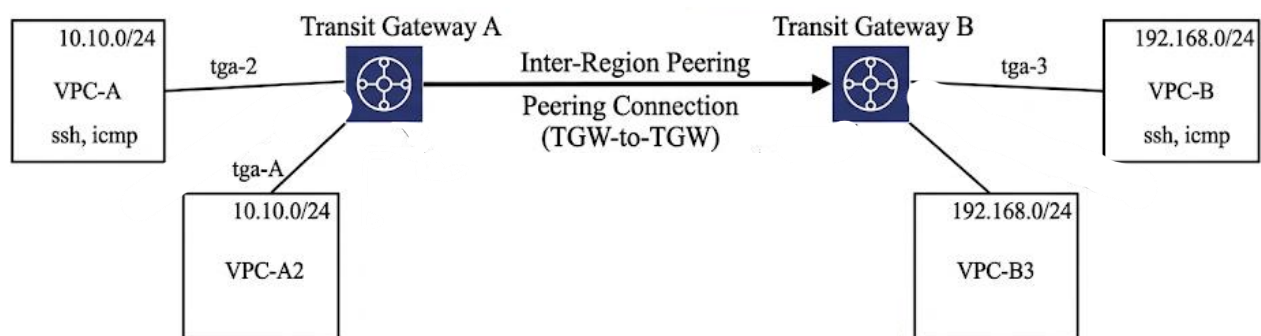
Important Limitations

- **No Transitive Peering:** If TGW-A is peered with TGW-B, and TGW-B is peered with TGW-C, TGW-A **cannot** talk to TGW-C through TGW-B. You must create a direct peering link between A and C.
- **Static Routes Only:** Routes across peering attachments must be added **statically**; they are not propagated automatically like VPC attachments.

Comparison Table

Feature	Same Region	Different Regions
Number of TGWs	1 Transit Gateway	2 Transit Gateways (one per region)
Connection Type	VPC Attachment	Transit Gateway Peering Attachment
Complexity	Simple (Hub-and-Spoke)	Medium (Hub-to-Hub)
Data Transfer	Standard data processing fees	Inter-region data transfer fees apply

Architecture of Peering connection between two different transit gateways

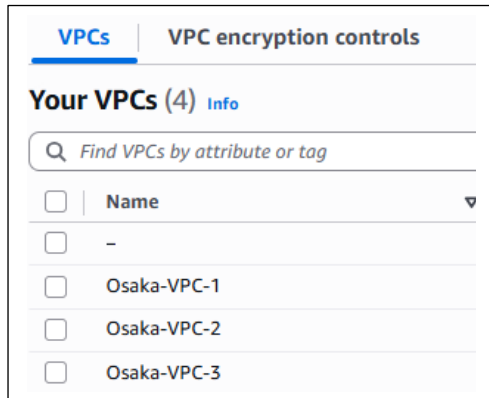


Hands on: Creating a peering connection between two different transit gateways in two different regions:

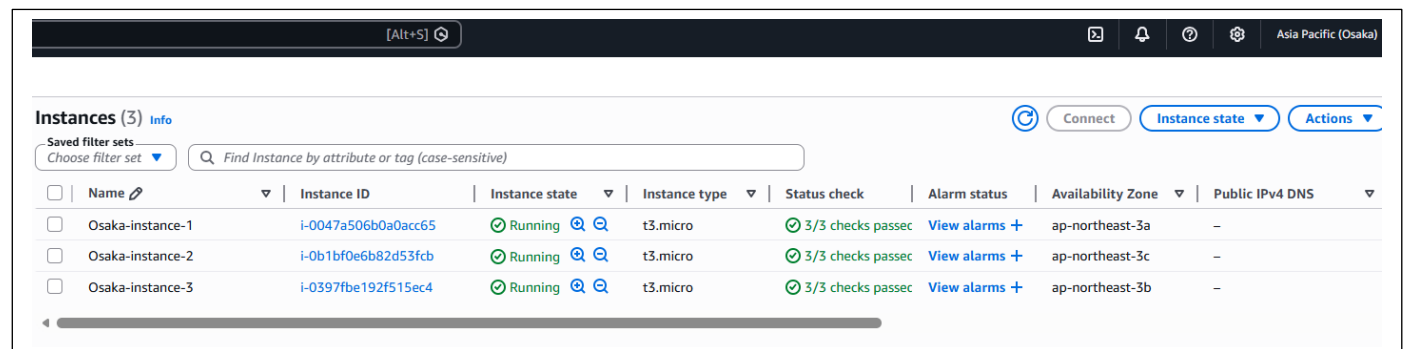
Step no:1 Create VPCs and Ec2 instances and make trsit gateway connection between them,

Step no:1.a Create three VPCs in the Osaka-region and three respective Ec2 instances for them. Make transit gateway communication between them using Transit gateway and transit gateway attachment.

VPC – Osaka region



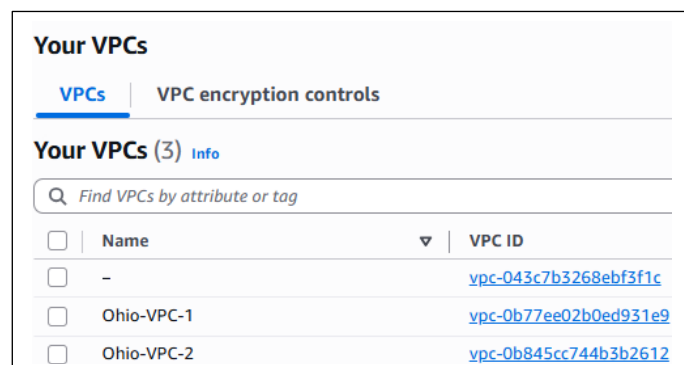
EC2 Instances- Osaka region



The transit gateway communication between them is established using Transit gateway and transit gateway attachment.

Step no:1.b Create three VPCs in the Ohio-region and three respective Ec2 instances for them. Make transit gateway communication between them using Transit gateway and transit gateway attachment.

VPC – Ohio region



EC2 Instances- Ohio region

Instances (2) Info									
Saved filter sets		Find Instance by attribute or tag (case-sensitive)							
Choose filter set									
☐	Name Info	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	Ohio-instance-2	i-0c5c31860345770b4	Running Q Q	t3.micro	Initializing	View alarms +	us-east-2b	-	13.58.165.61
☐	Ohio-instance-1	i-088019e15b22fd3b6	Running Q Q	t3.micro	Initializing	View alarms +	us-east-2a	-	3.147.205.245

The transit gateway communication between them is established using Transit gateway and transit gateway attachment.

Step no:02 In the VPC console of **Osaka Region** (Requestor), go to **Transit gateway Attachments** → Click **create** Transit gateway Attachments.

Name: **Osaka-Ohio-Connection-TGA**

Select: The **Existing Transit gateway ID** of the Osaka Region,
which is used for the communication between the VPCs inside that region.

Attachment Type: **Peering connection**

Connection Attachment: **My account**

Choose the region of the other VPC: **Ohio Region** we want to connect

Acceptor: **Paste Transit Gateway ID** of the Ohio Region

Click create transit gateway attachment

Create transit gateway attachment

Create transit gateway attachment [Info](#)

A transit gateway (TGW) is a network transit hub that interconnects attachments (VPCs and VPNs) within the same AWS account or across AWS accounts.

Details

Name tag - optional

Creates a tag with the key set to Name and the value set to the specified string.

Osaka-Ohio-Connection-TGA

Transit gateway ID [Info](#)

tgw-08e34ace1b43ae9de

Attachment type [Info](#)

Peering connection

Peering connection attachment

Select and configure your peering connection attachment.

Account [Info](#)

☒ My account

☐ Other account

Region [Info](#)

United States (Ohio) (us-east-2)

Transit gateway (accepter) [Info](#)

tgw-0da0d771572e22ed8

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q Osaka-Ohio-Connection-TGA

[Add new tag](#)

You can add up to 49 more tags.

[Cancel](#)

[Create transit gateway attachment](#)

It will show as initiating request in the Requestor region(ohio).

Transit gateway attachments (6) Info

Find transit gateway attachment by attribute or tag

☐	Name	Transit gateway attachment ID	Transit gateway ID	State	Resource type	Resource ID
☐	Osaka-Ohio-Connect...	tgw-attach-01c3de6d2b57bc5d1	tgw-08e34ace1b43ae9de	Initiating request	Peering	tgw-0da0d771572e22ed8
☐	Osaka-TGAttach...	tgw-attach-004f298d6ccc993b0	tgw-08e34ace1b43ae9de	Available	VPC	vpc-060f091e071a2448a

Step no:03 Go the VPC console of the **Ohio region** (Acceptor). → **Transit gateway** → we can see the **peering request from the Osaka region**. It will be in pending acceptance.

Transit gateway attachments (3) Info

Find transit gateway attachment by attribute or tag

☐	Name	Transit gateway attachment ID	Transit gateway ID	State	Resource type	Resource ID
☐		tgw-attach-01c3de6d2b57bc5d1	tgw-0da0d771572e22ed8	Pending acceptance	Peering	tgw-08e34ace1b43ae9de
☐	Ohio-TGA1	tgw-attach-09c5892908182047d	tgw-0da0d771572e22ed8	Available	VPC	vpc-0b77ee02b0ed931e9

Select → action → **accept** the transit gateway attachment

Transit gateway attachments (1/3) Info

Find transit gateway attachment by attribute or tag

☒	Name	Transit gateway attachment ID	Transit gateway ID	State	Resource type	Resource ID
☒		tgw-attach-01c3de6d2b57bc5d1	tgw-0da0d771572e22ed8	Pending acceptance	Peering	tgw-08e34ace1b43ae9de
☐	Ohio-TGA1	tgw-attach-09c5892908182047d	tgw-0da0d771572e22ed8	Available	VPC	vpc-0b77ee02b0ed931e9
☐	Ohio-TGA2	tgw-attach-0e58cf638b340d850	tgw-0da0d771572e22ed8	Available	VPC	vpc-0b845cc74b3b2612

Actions
Create flow log
Manage tags
Accept transit gateway attachment
Reject transit gateway attachment
Delete transit gateway attachment

Step no:04 Go to the VPC console of **Osaka Region** (Requestor).

Scroll down → **Transit gateway route table**.

We can see a **Transit gate way Route table** has been created for the Transit gateway. → select → **Routes** → **Create Static route**

The Local CIDR ranges of the VPCs will be there (11.0.0.0/16, 12.0.0.0/16,13.0.0.0/16).

Add the CIDR range of the VPCs in the acceptor region (Ohio)

15.0.0.0/16 and 14.0.0.0/16

Transit gateway route tables: tgw-rtb-04193d6c7e76fb86c

Filter routes by state

Exact CIDR
Select a valid IP4 or IPv6 CIDR.
0.0.0.0/0, ::/0

Longest prefix match
Enter a valid IP4 or IPv6 and press enter.
0.0.0.0, ::

Supernet of match
Select a valid IP4 or IPv6 CIDR.
0.0.0.0/0, ::/0

Routes (5) Info

Find route by attribute or tag

☐	CIDR	Attachment ID	Resource ID	Resource type	Route type
☐	11.0.0.0/16	tgw-attach-04f36ec4c1f0dc7c2	vpc-0ddca4ad66201f12b	VPC	Propagated
☐	12.0.0.0/16	tgw-attach-0bb3f0e00978231b7	vpc-0bec6c68daf715e9a	VPC	Propagated
☐	13.0.0.0/16	tgw-attach-004f298d6ccc993b0	vpc-060f091e071a2448a	VPC	Propagated
☐	14.0.0.0/16	tgw-attach-01c3de6d2b57bc5d1	tgw-0da0d771572e22ed8	Peering	Static
☐	15.0.0.0/16	tgw-attach-01c3de6d2b57bc5d1	tgw-0da0d771572e22ed8	Peering	Static

Step no:05 Go the VPC Route Table → Choose **My-RT-1** → Add the CIDR of the VPCs in the Ohio Region (14.0.0.0/16 , 15.0.0.0/16) → Transit gateway → our gateway ID → save changes.

Edit routes

Destination	Target	Status	Propagated	Route Origin	
11.0.0.0/16	local	Active	No	CreateRouteTable	
Q 12.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 13.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 0.0.0.0/0	Internet Gateway	Active	No	CreateRoute	Remove
Q 14.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove
Q 15.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove

[Add route](#)

[Cancel](#) [Preview](#) [Save changes](#)

Go the VPC Route Table → Choose **My-RT-2** → Add the CIDR of the VPCs in the Ohio Region (14.0.0.0/16 , 15.0.0.0/16) → Transit gateway → our gateway ID → save changes.

Edit routes

Destination	Target	Status	Propagated	Route Origin	
12.0.0.0/16	local	Active	No	CreateRouteTable	
Q 11.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 13.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 0.0.0.0/0	Internet Gateway	Active	No	CreateRoute	Remove
Q 14.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove
Q 15.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove

[Add route](#)

[Cancel](#) [Preview](#) [Save changes](#)

Go the VPC Route Table → Choose **My-RT-3** → Add the CIDR of the VPCs in the Ohio Region (14.0.0.0/16 , 15.0.0.0/16) → Transit gateway → our gateway ID → save changes.

Edit routes

Destination	Target	Status	Propagated	Route Origin	
13.0.0.0/16	local	Active	No	CreateRouteTable	
Q 11.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 12.0.0.0/16	Transit Gateway	Active	No	CreateRoute	Remove
Q 0.0.0.0/0	Internet Gateway	Active	No	CreateRoute	Remove
Q 14.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove
Q 15.0.0.0/16	Transit Gateway	-	No	CreateRoute	Remove

[Add route](#)

[Cancel](#) [Preview](#) [Save changes](#)

Step no:06 Go to the VPC console of Ohio Region (Acceptor).

Scroll down → **Transit gateway route table.**

We can see a Transit gateway Route table has been created for the Transit gateway. → select → Routes → **Create Static route**

The Local CIDR ranges of the VPCs will be there (14.0.0.0/16, 15.0.0.0/16).

Add the CIDR range of the VPCs in the Requestor region (Osaka)

11.0.0.0/16, 12.0.0.0/16 and 13.0.0.0/16

Transit gateway route tables: tgw-rtb-0a7f79bdec7791d00

Q 0.0.0.0/0, ::/0 Q 0.0.0.0, :: Q 0.0.0.0/0, ::/0

Routes (5) Info

Q Find route by attribute or tag

☐	CIDR	Attachment ID	Resource ID	Resource type	Route type
☐	11.0.0.0/16	tgw-attach-01c3de6d2b57bc5d1	tgw-08e34ace1b43ae9de	Peering	Static
☐	12.0.0.0/16	tgw-attach-01c3de6d2b57bc5d1	tgw-08e34ace1b43ae9de	Peering	Static
☐	13.0.0.0/16	tgw-attach-01c3de6d2b57bc5d1	tgw-08e34ace1b43ae9de	Peering	Static
☐	14.0.0.0/16	tgw-attach-09c5892908182047d	vpc-0b77ee02b0ed931e9	VPC	Propagated
☐	15.0.0.0/16	tgw-attach-0e58cf638b340d850	vpc-0b845cc744b3b2612	VPC	Propagated

Step no:07 Go the VPC Route Table → Choose Ohio- Route table -1 → Add the CIDR of the VPCs in the Osaka Region

(11.0.0.0/16,12.0.0.0/16 , 13.0.0.0/16) → Transit gateway → our gateway ID → save changes.

Edit routes

Destination	Target	Status	Propagated	Route Origin
14.0.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Internet Gateway	Active	No	CreateRoute
Q 11.0.0.0/16	Transit Gateway	-	No	CreateRoute
Q 12.0.0.0/16	Transit Gateway	-	No	CreateRoute
Q 13.0.0.0/16	Transit Gateway	-	No	CreateRoute

Q local Q tgw-004f4b0c129133692 Q tgw-0da0d771572e22ed8 Q tgw-0da0d771572e22ed8 Q tgw-0da0d771572e22ed8

Add route

Cancel Preview Save changes

(11.0.0.0/16, 12.0.0.0/16, 13.0.0.0/16) → Transit gateway → **our gateway ID** → save changes.

Edit routes

Step no: 8 All the connections and configurations have been made. Now we want to check whether the EC2 instance & VPC in the Osaka region can communicate / connect with the EC2 instance & VPC in the Ohio region.

Ping the private ip of the Ec2 instance in the Ohio Region.

```
aws | Search [Alt+S]
#
~\#### Amazon Linux 2023
~\#####
~\###|
~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~V~'~>
~~
~-.-
~/m/'

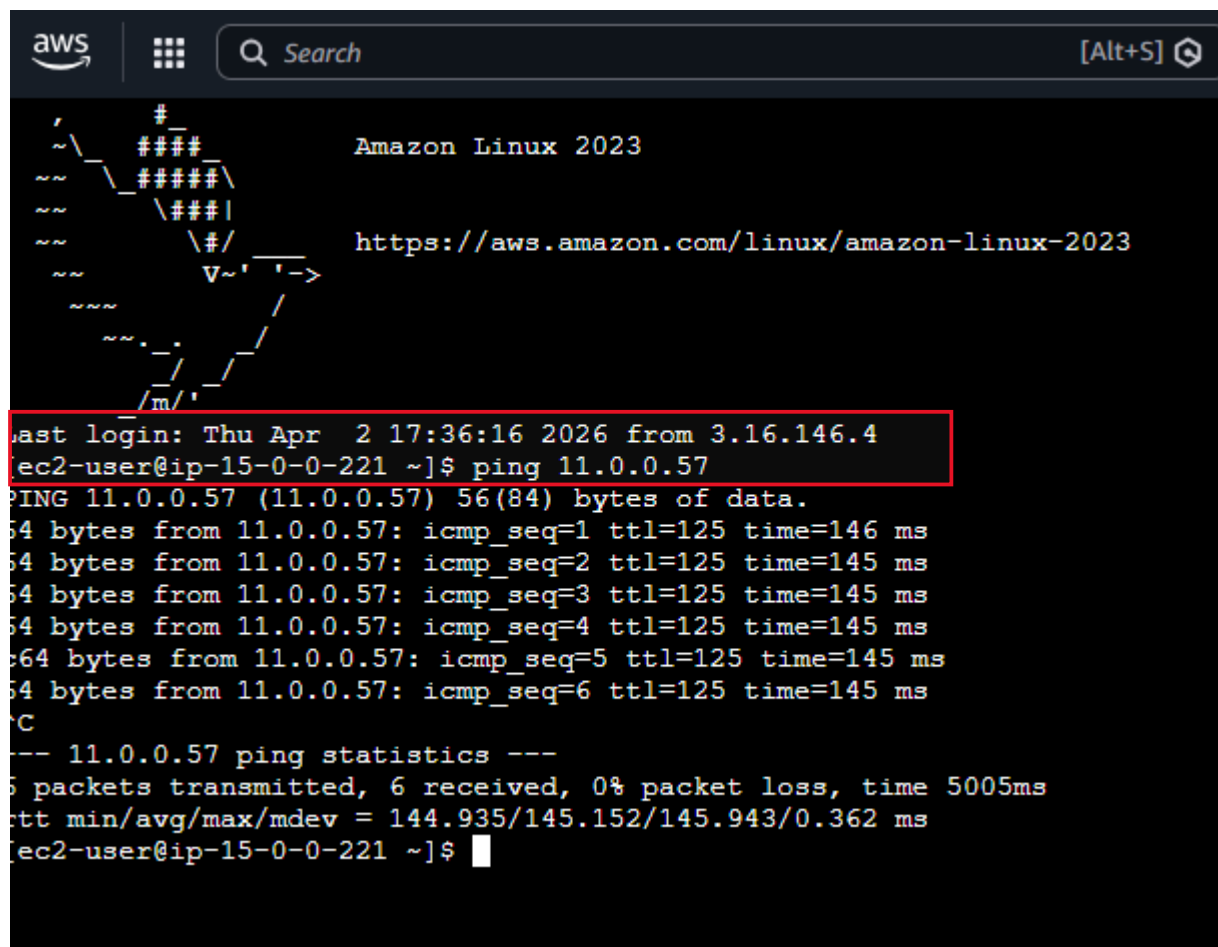
Last login: Thu Apr  2 17:35:38 2026 from 15.168.105.162
[ec2-user@ip-11-0-0-57 ~]$ ping 15.0.0.221
PING 15.0.0.221 (15.0.0.221) 56(84) bytes of data:
64 bytes from 15.0.0.221: icmp_seq=1 ttl=125 time=146 ms
64 bytes from 15.0.0.221: icmp_seq=2 ttl=125 time=145 ms
64 bytes from 15.0.0.221: icmp_seq=3 ttl=125 time=145 ms
64 bytes from 15.0.0.221: icmp_seq=4 ttl=125 time=145 ms
^C
--- 15.0.0.221 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 144.585/144.942/145.968/0.592 ms
[ec2-user@ip-11-0-0-57 ~]$
```

i-0047a506b0a0acc65 (Osaka-instance-1)

PublicIPs: 15.168.164.154 PrivateIPs: 11.0.0.57

Similarly, Go to **Ohio EC2 console** → click on Ec2 instance 1 → connect.

Ping the private ip of the Ec2 instance in the Osaka Region.

A screenshot of an AWS terminal window. The terminal shows the prompt 'ec2-user@ip-15-0-0-221 ~]' and the command 'ping 11.0.0.57'. The output shows six successful ping requests with 56(84) bytes of data, each taking approximately 145 ms. The ping statistics show 6 packets transmitted, 6 received, 0% packet loss, and a total time of 5005ms. The terminal also displays the 'last login' information and the URL 'https://aws.amazon.com/linux/amazon-linux-2023'.

```
aws | Search [Alt+S]
#
~\#### Amazon Linux 2023
~~\#####\
~~\####\
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~v~'-'>
~~~
~~.-.
~~/_/_/
/m/'-'

last login: Thu Apr 2 17:36:16 2026 from 3.16.146.4
ec2-user@ip-15-0-0-221 ~]$ ping 11.0.0.57
PING 11.0.0.57 (11.0.0.57) 56(84) bytes of data.
64 bytes from 11.0.0.57: icmp_seq=1 ttl=125 time=146 ms
64 bytes from 11.0.0.57: icmp_seq=2 ttl=125 time=145 ms
64 bytes from 11.0.0.57: icmp_seq=3 ttl=125 time=145 ms
64 bytes from 11.0.0.57: icmp_seq=4 ttl=125 time=145 ms
64 bytes from 11.0.0.57: icmp_seq=5 ttl=125 time=145 ms
64 bytes from 11.0.0.57: icmp_seq=6 ttl=125 time=145 ms
^C
--- 11.0.0.57 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5005ms
rtt min/avg/max/mdev = 144.935/145.152/145.943/0.362 ms
ec2-user@ip-15-0-0-221 ~]$
```

As we ping the private ip of the instances interchangeably in different region and it works, it shows the Peering communication is successful.